

Constructing a Command-Line Shell: A Systems-Level Theoretical Guide

Section 1: The Shell as a User-Kernel Interface

1.1 Defining the Command-Line Shell

A command-line shell is a sophisticated computer program that serves as the primary text-based interface between a user and the operating system's kernel.¹ In its most fundamental form, it is a command-line interpreter, a specialized application designed to read lines of text entered by a user, parse them to understand the user's intent, and then execute the corresponding commands.³ This process of interaction is often modeled as a Read-Eval-Print Loop (REPL), a continuous cycle where the shell reads input, evaluates it by invoking system services, prints the resulting output, and then loops back to await the next command.⁵ This model forms the architectural heartbeat of any shell, from the most basic implementation to feature-rich versions like

bash or zsh.

The shell's core function is to translate high-level, human-readable commands into low-level system calls that the operating system kernel can understand and act upon.⁵ For instance, when a user types a simple command like

ls to list directory contents, the shell is responsible for locating the ls program on the filesystem, creating a new process to run it, and displaying its output back to the user. This act of translation and orchestration is the shell's essential value; it provides a powerful layer of abstraction, hiding the immense complexity of direct kernel interaction and making the operating system's vast capabilities accessible and manageable.¹ By building a shell, one moves from being a mere user of this abstraction to becoming its architect, gaining a profound understanding of how user intent is transformed into system action.

1.2 The Shell's Place in the Linux Architecture

To fully grasp the shell's role, it is essential to visualize its position within the layered architecture of a typical Linux system. This architecture is generally conceptualized as a hierarchy of layers, each building upon the one below it: Hardware, Kernel, Shell, and Applications.⁹

At the base is the **Hardware**, encompassing the physical components of the computer: the CPU, RAM, storage devices, and peripherals.⁷ Directly interfacing with the hardware is the

Kernel, the central core of the operating system.⁹ The kernel is a privileged program responsible for managing all hardware resources, scheduling processes, handling memory allocation, and providing a consistent set of services to higher-level software through an interface known as system calls.⁷

The **Shell** operates in the layer just above the kernel. It is a user-space application, meaning it runs with the same privileges as any other program a user might launch, and is not an intrinsic part of the kernel itself.⁷ This is a critical distinction; the shell is a replaceable component of the user environment, not a monolithic part of the OS. Its purpose is to provide an interactive gateway to the kernel's services. When a user issues a command, the shell interprets it and makes the necessary system calls to the kernel, which then performs the requested operation.⁷

Finally, at the top layer are the **Applications**, which are the programs users interact with directly, such as web browsers, text editors, and the command-line tools the shell itself executes.¹⁰ It is also important to distinguish the shell from the

Terminal. The terminal (or terminal emulator) is the graphical application that provides the text-based window and keyboard input (e.g., `gnome-terminal`, `xterm`). The shell is the command interpreter program that runs *inside* that terminal window, processing the input and generating the output that the terminal displays.⁵ Understanding that the shell is a standard user-space program demystifies the task of building one; it is not an exercise in kernel modification, but rather an advanced application development project that leverages the powerful API provided by the kernel.

1.3 A Taxonomy of Shells and Commands

The concept of a command-line shell is not monolithic; numerous implementations have been developed over the history of Unix-like systems, each with different features and scripting syntax. The earliest widely used shell was the Bourne Shell (sh), known for its simplicity and power in scripting.¹ Later, shells like the C Shell (

csh) introduced features more familiar to C programmers, such as command history and job control.³ The Korn Shell (

ksh) combined features from both, and modern systems predominantly use the Bourne-Again Shell (bash) or the Z Shell (zsh), which are highly advanced, POSIX-compliant shells that incorporate decades of feature development.¹ Building a new shell means creating another entry in this long lineage, typically starting with a subset of the core, POSIX-defined functionality.

A final crucial distinction is between the shell program itself and the commands it executes. Most commands a user types, such as ls, grep, cat, or gcc, are not part of the shell. They are separate, standalone executable programs located on the filesystem.⁵ The shell's job is to find these programs and run them on the user's behalf. However, some commands, like

cd and exit, are implemented directly within the shell's code; these are known as "built-in" commands.² The reasons for this dual approach to command execution are fundamental to the operating system's process model and will be a central topic of this guide.

Section 2: Architectural Blueprint and Incremental Development

2.1 Structuring a C Project for a Systems Application

A well-organized project structure is paramount for managing the complexity of a systems-level application like a shell. Adopting a modular approach from the outset promotes clarity, maintainability, and scalability. A recommended structure separates different components of the project into distinct directories, clarifying the boundary between interfaces and their implementations.¹⁶

A standard and effective layout includes the following directories:

- **src/:** This directory will house all C source code files (.c files). Logic should be separated into modules based on concern, for example: main.c for the main REPL, parser.c for command parsing logic, execute.c for command execution, and builtins.c for built-in command implementations.¹⁶
- **include/:** This directory contains all the corresponding header files (.h files). Each header file should define the public Application Programming Interface (API)—function prototypes, type definitions, and macros—for its corresponding source module. This separation allows modules to interact through well-defined interfaces without needing to know the details of each other's implementation.¹⁶
- **bin/:** This directory is designated for the final compiled executable binary. Keeping build artifacts separate from source code is a clean practice.
- **Makefile:** A crucial file at the root of the project that contains the instructions for the make utility to build the entire project from its source files.

This separation of concerns is a cornerstone of robust software engineering. It not only makes the codebase easier to navigate but also facilitates independent development and testing of different components.

2.2 The Role of the Makefile

The Makefile is the blueprint for the project's build process. It is a script read by the make utility, which automates the complex and often repetitive tasks of compiling source code into object files and linking those object files into a final executable.¹⁷ A

Makefile is composed of rules, each of which defines how to create a specific file, known as a target.¹⁸

The core components of a rule are:

- **Target:** The file to be created, such as an object file (main.o) or the final executable (myshell).¹⁸
- **Prerequisites (Dependencies):** A list of files that the target depends on. For an object file, the prerequisites are its source (.c) and any header (.h) files it includes. For an executable, the prerequisites are all the object files that need to be linked together.²⁰
- **Commands:** A sequence of shell commands (e.g., gcc, ld) that are executed to create the target from its prerequisites. Crucially, these command lines must be indented with a tab character, not spaces.¹⁸

The primary advantage of using make is its ability to perform intelligent, incremental builds. make examines the modification timestamps of targets and their prerequisites. It will only

execute the commands to rebuild a target if the target file does not exist or if any of its prerequisites have been modified more recently than the target itself.¹⁸ For a large project, this saves an enormous amount of time by avoiding the unnecessary recompilation of unchanged code. More than a mere convenience, a well-written

Makefile serves as a formal declaration of the project's dependency graph. It forces the developer to think explicitly about the relationships between modules, thereby reinforcing a disciplined and well-architected project structure.

2.3 A Pedagogy of Incremental Development

Given the intricate interplay of system calls and process management involved in building a shell, a monolithic development approach is fraught with peril. An incremental development strategy—building and testing small, self-contained pieces of functionality one at a time—is strongly recommended.²¹ This methodology dramatically reduces the complexity of debugging by isolating potential errors to the most recently added code. It also provides a series of rewarding milestones, which is psychologically beneficial for a complex project.²²

This incremental path can be designed to mirror the historical evolution of shells themselves, starting with a simple core and progressively adding layers of sophistication. This is not merely a good coding practice; it is a pedagogical journey that follows the logical problem-solving path of the original Unix developers, providing a deeper context for the development process. A logical sequence for building a shell is as follows:

1. **Core Loop:** Begin by implementing the basic REPL structure. The program should be able to display a prompt, read a line of input from the user, and print it back. This establishes the fundamental interactive loop.²³
2. **Parsing:** Integrate the parsing logic. The shell should now be able to take the input line and tokenize it into a command and its arguments, storing them in a suitable data structure.²³
3. **External Command Execution:** Implement the core fork/execvp/waitpid cycle. At this stage, the shell becomes truly functional, capable of executing external programs like ls, pwd, or echo.²³
4. **Built-in Commands:** Add the internal logic to recognize and handle built-in commands. Start with exit to allow for a clean shutdown, followed by cd to manage the working directory.²⁶
5. **I/O Redirection:** Implement output (>) and input (<) redirection by manipulating file descriptors within the child process before the execvp call.²⁷
6. **Pipes:** Extend the I/O manipulation logic to support piping (|) between two or more commands, which involves creating a pipe and coordinating multiple child processes.²⁷

7. **Signal Handling:** Finally, add signal handlers to gracefully manage user interruptions like Ctrl+C (SIGINT) and Ctrl+Z (SIGTSTP), making the shell more robust and interactive.

Section 3: The Heartbeat of the Shell: The Read-Eval-Print Loop (REPL)

3.1 Deconstructing the REPL Model

The fundamental operational paradigm of a command-line shell is the Read-Eval-Print Loop (REPL).⁵ This simple yet powerful model describes a continuous, four-phase cycle that defines the shell's interactive lifetime.⁶ While the model itself is straightforward, its "Eval" phase encapsulates the entirety of the shell's complex functionality. This provides a potent mental model for development: the entire project can be viewed as a progressive and layered refinement of the evaluation step within a simple, infinite loop.

The four phases are:

1. **Read:** The shell blocks and waits for the user to enter a command at the prompt. It must read an entire line of text from its standard input stream.²³ A significant technical challenge in C is reading a line of arbitrary length, as one cannot know in advance how much the user will type. Statically allocating a buffer risks overflow if the input is too long. The modern and recommended solution is to use the POSIX `getline()` function, which automatically handles dynamic memory allocation, resizing the buffer as needed to accommodate the entire line.²⁴
2. **Eval(uate):** This is the core of the shell's logic. Once a line of input is successfully read, the shell must evaluate it. This is not a single action but a sequence of complex operations that form the subject of the subsequent sections of this guide. Evaluation involves parsing the command string into tokens, interpreting special characters for redirection or piping, determining if the command is a built-in or an external program, and finally, executing the command by either calling an internal function or orchestrating the creation of new processes.²³
3. **Print:** After the command has been evaluated and executed, any output it generated will have been displayed on the screen (standard output), along with any potential error messages (standard error). The "Print" phase in the REPL model refers to the shell ensuring that this output is delivered to the user before the loop continues.
4. **Loop:** The shell returns to the beginning of the cycle, prints a new prompt to signal its

readiness, and waits for the next user command. This loop continues indefinitely until a specific termination condition is met.²³

3.2 Initializing the Environment

Before entering the main REPL, a shell performs an initialization phase.²³ In sophisticated shells like

bash, this involves reading and executing configuration files (e.g., `.bashrc`) to set up the user's environment, define aliases, and configure shell behavior. For a basic shell built from scratch, this phase is simpler but still conceptually important. It is the logical place to perform one-time setup tasks, such as:

- Initializing data structures.
- Determining if the shell is running in an interactive mode (connected to a terminal).
- Setting up initial signal handlers to ensure the shell itself responds correctly to interruptions from the outset.

3.3 Terminating the Shell

The REPL is not truly infinite; it must have a clean exit path. The termination phase is triggered when the loop condition is broken, which can happen in two primary ways:

1. **Explicit Command:** The user enters a built-in exit command. The evaluation phase recognizes this command and sets a flag to terminate the loop.
2. **End-of-File (EOF):** The user signals the end of input, typically by pressing Ctrl+D on an empty line. The read operation (e.g., `getline()`) will return a special value (typically -1) indicating EOF, which the loop must detect to terminate.²⁴

Upon deciding to terminate, the shell must perform cleanup. This is a critical step to ensure it is a well-behaved program. The primary task is to free any and all dynamically allocated memory, such as the buffer used for reading the line and the data structures created during parsing, to prevent memory leaks.²³ After cleanup, the shell exits, returning a status code to its parent process.

Section 4: Deconstructing Commands: Parsing and Tokenization Strategies

4.1 The Goal of Parsing

The raw string of characters a user types into the prompt, such as `grep -i "error" log.txt > results.txt`, is a high-level expression of intent. For the operating system's execution machinery, which deals in structured arguments and system calls, this string is opaque and meaningless. The fundamental purpose of the parsing stage is to bridge this gap. Parsing is the process of lexical and syntactic analysis that transforms the unstructured character string into a structured, hierarchical data representation that the shell's execution engine can unambiguously interpret and act upon.³¹

The output of this process must clearly delineate several key components: the executable command, its associated flags and arguments, and any special operators that modify the command's execution environment, such as those for input/output redirection (`<`, `>`) or inter-process communication (`|`).³² This transformation from a raw string to a structured command is the first and most critical step in the "Eval" phase of the REPL.

4.2 Tokenization: The First Step

The initial phase of parsing is tokenization (or word splitting), which involves breaking the input string into a sequence of meaningful substrings called tokens.³⁶ In the context of a simple shell, the primary delimiters that separate tokens are whitespace characters: spaces and tabs.²⁵ For example, the command

`ls -l /home` would be tokenized into three distinct strings: `"ls"`, `"-l"`, and `"/home"`.

A common and straightforward tool for this task in the C standard library is the `strtok()` function.³¹ This function is designed to be called iteratively to extract successive tokens from a string. The usage pattern is distinct:

- The first call to `strtok()` is given the string to be tokenized and a string containing the set of delimiter characters. It finds the first token, internally saves a pointer to the next

- position in the string, and returns a pointer to the beginning of the token.³⁶
- Subsequent calls to `strtok()` must pass `NULL` as the first argument. This signals to the function to continue tokenizing the *same string* from where it left off in the previous call. It will continue to return pointers to new tokens until no more are found, at which point it returns `NULL`.³⁸

4.3 The Caveats of `strtok()`

While `strtok()` is convenient, it comes with critical caveats that must be understood to be used correctly. First and foremost, `strtok()` is a destructive function. As it identifies tokens, it modifies the original input string by replacing the delimiter character that follows each token with a null terminator (`\0`).³⁷ This is how it creates distinct, null-terminated C-strings for each token within the original buffer. The consequence is that the original input string is destroyed in the process. If the original command line is needed for any other purpose (e.g., for a history feature), a copy must be made before passing it to

`strtok()`.

Secondly, the standard `strtok()` function is not reentrant or thread-safe. It uses a hidden, internal static pointer to keep track of its position within the string between calls. This means that if `strtok()` is used in two different threads on two different strings simultaneously, or if a signal handler that also uses `strtok()` interrupts a `strtok()` loop, the internal state will be corrupted, leading to unpredictable behavior. For a simple, single-threaded shell, this is not an immediate concern. However, it is good practice to be aware of the reentrant alternative, `strtok_r()`, which requires the caller to provide a pointer to store the state, making it safe for concurrent use.⁴⁰

The choice of parsing technique directly dictates the syntactic power of the shell. A simple `strtok()`-based parser is sufficient for basic commands but inherently struggles with more complex shell syntax, such as handling quoted arguments (e.g., treating "hello world" as a single token) or variable expansion. Implementing these features would require a more sophisticated, custom-written parser that can manage state (e.g., "am I inside a quote?"), rather than relying on simple delimiter splitting. The initial choice of `strtok()` is thus a pragmatic trade-off, prioritizing simplicity of implementation over syntactic richness.

4.4 Data Structure for Parsed Commands

The ultimate goal of the parsing process is to produce a data structure that is directly consumable by the `execvp()` system call, which is used to execute external programs. This system call expects the command and its arguments to be passed as a NULL-terminated array of character pointers (`char *argv` or `char **argv`).⁴¹

Therefore, the parser's output should be a dynamically allocated array of `char*`. The structure should be as follows:

- `argv` points to the token representing the command itself (e.g., `"ls"`).
- `argv`, `argv`, etc., point to the tokens for each subsequent argument (e.g., `"-l"`, `"/home"`).
- The final element of the array must be a NULL pointer. This NULL acts as a sentinel, marking the end of the argument list for `execvp()`.³¹

This transformation of an unstructured string into a formal `argv` array is the successful completion of the parsing stage, preparing the command for execution.

Section 5: Process Genesis and Metamorphosis: The `fork()` and `exec()` Dance

5.1 The `fork()` System Call: Creating a New Process

The fundamental mechanism for creating new processes in Unix and Linux is the `fork()` system call.⁴³ Its behavior is unique and powerful: when a process (the "parent") calls

`fork()`, the kernel creates a new process (the "child") that is an almost exact duplicate of the parent.²³ This duplication includes a copy of the parent's virtual address space (its code, data, and stack), its open file descriptors, and its environment variables.⁴³ The parent and child processes then continue execution concurrently from the instruction immediately following the

`fork()` call.⁴⁵

The most critical aspect of `fork()` is its return value, which allows the program to differentiate between the two otherwise identical processes⁴³:

- In the **child process**, `fork()` returns 0.
- In the **parent process**, `fork()` returns the Process ID (PID) of the newly created child. The

PID is a unique positive integer.

- If the process creation fails (e.g., due to resource limits), `fork()` returns -1 to the parent, and no child process is created.

This divergent return value is the linchpin of process management. By checking if the return value is zero, positive, or negative, the code can follow different execution paths for the parent and the child, allowing them to perform different tasks.⁴⁵

5.2 The `exec()` Family: Transforming a Process

Creating a duplicate process with `fork()` is only half the story. To run a different program, the child process must undergo a metamorphosis using a function from the `exec()` family.³¹ The

`exec()` system calls replace the current process's entire memory image—its code, data, and stack segments—with a new program loaded from an executable file on disk.⁴¹ This transformation is total; if the

`exec()` call is successful, the original program code is gone forever, and execution begins at the entry point of the new program. Consequently, a successful `exec()` call *never returns* to the calling code.³³ It only returns (with a value of -1) if an error occurs, such as the specified program not being found.

For building a shell, the `execvp()` variant is particularly suitable for two reasons⁴¹:

1. The `v` in `execvp` stands for "vector," meaning it accepts the new program's arguments as a NULL-terminated array of strings (`char *argv`). This format perfectly matches the output of the command parser described in the previous section.
2. The `p` in `execvp` stands for "path." If the filename argument does not contain a slash (`/`), `execvp` will automatically search for the executable in the list of directories specified by the `PATH` environment variable.³² This behavior emulates how real shells find and execute commands like `ls` or `git` without requiring the user to type their full path (e.g., `/bin/ls`). This leverages a critical system-wide convention, effectively treating the `PATH` variable as a namespace for all available command-line executables.

5.3 The Fundamental Execution Pattern

The combination of `fork()` and `execvp()` forms the canonical pattern for running external commands in a Unix shell.²⁴ This "clone and replace" paradigm is the foundation of multitasking on the platform. The two-step process provides a crucial window of opportunity: after

`fork()` but before `execvp()`, the child process exists as a clone of the parent and can perform setup operations. It can manipulate its environment, and most importantly, its file descriptors, to prepare for the new program it is about to become. This capability is what enables powerful features like I/O redirection and piping, as the child can re-wire its own standard input and output before handing over control to the new program.

The sequence of operations is as follows:

1. The parent shell receives and parses a command.
2. It calls `fork()` to create a child process.
3. The code immediately checks the return value of `fork()`.
4. **If in the child process (return value is 0):**
 - The child performs any necessary setup (e.g., I/O redirection, to be discussed later).
 - It calls `execvp()` with the parsed command and arguments.
 - If `execvp()` fails, it is critical for the child to print an error message (e.g., "command not found") and then terminate immediately using `exit()`. Failure to exit would leave a confused clone of the shell running, which could then attempt to continue the parent's work.
5. **If in the parent process (return value is the child's PID):**
 - The parent process typically calls `waitpid()` to suspend its own execution until the child process completes. This ensures that commands are executed sequentially and the prompt is only displayed after the previous command has finished.

Section 6: Command Execution: Differentiating Built-ins from External Programs

6.1 Defining Built-in and External Commands

The command execution logic within a shell follows a crucial bifurcation. After parsing a user's command, the shell must decide upon one of two distinct execution pathways. This decision is based on whether the command is **external** or **built-in**.¹⁴

- **External Commands** are standalone executable programs that reside in the filesystem, typically in directories like `/bin`, `/usr/bin`, or `/usr/local/bin`.¹⁴ Commands like `ls`, `grep`, `gcc`, and `find` fall into this category. The shell executes these by employing the fork-exec pattern detailed in the previous section: it creates a new child process, and that child process transforms itself into the desired program.¹⁵
- **Built-in Commands** are not separate programs. Their logic is implemented directly as functions within the shell's own source code.¹ When a user invokes a built-in command, the shell executes it by simply calling the corresponding internal function. No new process is created via `fork()`, and no external program is loaded via `exec()`.²

This distinction means the core of the shell's execution engine is a decision tree. After parsing the command, the first logical step is to check if the command name matches one of the known built-ins. If it does, the shell calls the relevant internal C function. If not, it proceeds with the fork-exec mechanism for external commands.

6.2 The Rationale for Built-ins: Modifying the Shell's State

The necessity for built-in commands is not an arbitrary design choice but a direct and fundamental consequence of the process isolation model inherent to Unix-like operating systems. When `fork()` creates a child process, the child receives a *copy* of the parent's memory space and environment, not a reference to it.⁴⁵ This isolation is a critical feature for system stability and security; it prevents a child process from maliciously or accidentally corrupting its parent's state.

The canonical example that illustrates this principle is the `cd` (change directory) command.⁴⁹ Every process maintains its own "current working directory" (CWD), which is an attribute of the process's state managed by the kernel. If

`cd` were an external program (e.g., `/bin/cd`), executing it would follow this sequence:

1. The parent shell would `fork()` a child process.
2. The child process would `exec()` the `/bin/cd` program.
3. The `/bin/cd` program, running inside the child process, would successfully call the `chdir()` system call to change *its own* current working directory.
4. The `/bin/cd` program would then finish its task and exit immediately.
5. The parent shell, which had been waiting, would resume execution, its own current working directory having been completely unaffected by the child's actions.⁴⁹

The change would be lost the moment the child process terminated. Therefore, any command

that must fundamentally alter the state of the shell itself—its working directory, its environment variables (export), its aliases (alias), or its very existence (exit)—*must* be implemented as a built-in command. By implementing cd as a function within the shell, the call to the chdir() system call occurs directly within the shell's process, correctly modifying its own CWD.³¹

6.3 Other Built-ins and Performance

Beyond the necessity of state modification, there is a secondary reason for implementing certain simple commands as built-ins: performance. The process of forking and executing a new program involves significant overhead, including memory allocation, copying page tables, and loading a new binary from disk. For very simple and frequently used commands (e.g., echo, pwd), implementing them as built-ins avoids this overhead, resulting in noticeably faster execution.¹⁴ A comprehensive list of common built-in commands found in shells like

csh includes alias, bg, fg, history, jobs, kill, setenv, and exit, among many others.⁵²

The following table provides a concise summary of the critical distinctions between these two command types.

Characteristic	Built-in Command (cd, exit)	External Command (ls, grep)
Execution Context	Runs within the shell's own process.	Runs in a new, separate child process.
Process Creation	No fork() call is made.	Requires a fork() and execvp() cycle.
State Modification	Can directly modify the parent shell's state (e.g., CWD, environment).	Cannot modify the parent shell's state due to process isolation.
Performance	Faster; execution is a direct function call with no process creation overhead.	Slower; involves the overhead of process creation and program loading from disk.

Section 7: Managing the Progeny: Process Synchronization and the Undead

7.1 The Need for Synchronization: Reaping Child Processes

The `fork()` system call initiates concurrent execution; after the call, both the parent and child processes are scheduled to run by the operating system.⁴³ For an interactive shell running a command in the foreground, this concurrency is undesirable. The standard behavior is for the shell to pause and wait for the command to complete before displaying a new prompt and accepting further input.¹ This requires a mechanism for process synchronization.

This synchronization is achieved through the `wait()` family of system calls, primarily `wait()` and `waitpid()`.³¹ When a parent process calls one of these functions, its execution is suspended (blocked) by the kernel until one of its child processes changes state, most commonly by terminating.⁵³ Once a child terminates, the parent is unblocked, the

`wait()` call returns, and the parent can resume its execution—for a shell, this means printing the next prompt. This act of waiting for and acknowledging a child's termination is known as "reaping" the child.

7.2 `wait()` vs. `waitpid()`

The C library provides two primary functions for reaping children, which offer different levels of control:

- `wait(int *statloc)`: This is the simpler of the two. It suspends the calling process until *any* of its child processes terminates. It returns the PID of the terminated child, and if the `statloc` argument is not NULL, it stores the child's exit status information at that memory location.⁵³ Its simplicity is also its limitation; the parent has no control over which child it waits for.
- `waitpid(pid_t pid, int *statloc, int options)`: This function is a more powerful and flexible superset of `wait()`. The `pid` argument allows the caller to specify which child or group of

children to wait for.⁵³

- If `pid > 0`, it waits for the specific child with that Process ID.
- If `pid == -1`, it waits for any child, making it equivalent to `wait()`.
- The options argument provides further control. Most notably, the `WNOHANG` option makes the call non-blocking. If a child has not terminated, `waitpid()` with `WNOHANG` will return 0 immediately instead of blocking, allowing the parent to poll for child status while performing other work.⁵³

For a simple shell that only handles one foreground command at a time, `wait()` is sufficient. However, `waitpid()` is the more robust choice, as its ability to wait for a specific PID and its non-blocking option are the essential primitives required to build more advanced features like job control (managing multiple background processes).

7.3 The Zombie Process: A Feature, Not a Bug

The term "zombie process" often sounds like an error condition, but it describes a normal and necessary intermediate state in a process's lifecycle. When a process terminates, the kernel deallocates most of its resources (like memory and CPU context), but it retains the process's entry in the system-wide process table.⁵⁶ This entry holds crucial information for the parent: the child's PID and its exit status. A process in this terminated-but-not-yet-reaped state is called a zombie.⁵⁶

This mechanism functions as a form of reliable, asynchronous communication. A child can terminate at any time, and its parent might not be ready to immediately process that event. The zombie state acts as a message buffer, where the kernel holds the "message" (the exit status) until the parent is ready to receive it by calling `wait()` or `waitpid()`.⁵⁶ The act of reaping—the

`wait()` call—is what reads this message. Once the parent has retrieved the exit status, the kernel's job is done, and it can finally remove the process table entry, allowing the PID to be reused. The problem associated with zombies is not their existence, but the failure of a parent process to reap them, which leads to a leak of process table slots—a finite system resource.

7.4 The Orphan Process: A Child Without a Parent

The inverse scenario occurs when a parent process terminates before one or more of its child processes. In this case, the still-running children become "orphan processes".⁵⁶ A

well-behaved operating system cannot simply leave these processes unmanaged, as they would become zombies upon termination with no parent to reap them.

To solve this, the kernel performs a process of "re-parenting." When the kernel detects that a process has become an orphan, it automatically changes the process's parent to be a special system process, known as the init process, which always has a PID of 1.⁵⁶ The

init process is the ancestor of all user-space processes on the system and is programmed to run a loop that continuously calls `wait()`, thereby ensuring that it reaps all of its adopted orphan children, preventing them from becoming permanent zombies.

Section 8: Orchestrating I/O: The Art of Redirection with File Descriptors and `dup2()`

8.1 The Unix Philosophy: "Everything is a File"

A core design principle of Unix-like operating systems is the abstraction that treats disparate I/O resources—such as files on a disk, pipes, network sockets, and terminal devices—as if they were all files.⁵⁹ This elegant simplification means that a small, consistent set of system calls (

`open`, `read`, `write`, `close`) can be used to interact with a wide variety of resources. The handle that a process uses to refer to one of these open "files" is a **file descriptor**: a simple, non-negative integer.⁶⁰

When a process wants to access a file or resource, it asks the kernel to open it. The kernel, in turn, adds an entry to a system-wide open file table and provides the process with a file descriptor, which is an index into the process's own private file descriptor table. This per-process table maps the integer descriptor to the corresponding entry in the system-wide table.⁶¹

8.2 The Three Standard Streams

By convention, every process inherits three pre-opened file descriptors from its parent when

it is created ⁶¹:

- **File Descriptor 0:** Standard Input (stdin). This is the default source of input for a program.
- **File Descriptor 1:** Standard Output (stdout). This is the default destination for normal output.
- **File Descriptor 2:** Standard Error (stderr). This is the default destination for error messages.

For an interactive shell, all three of these streams are initially connected to the user's terminal, allowing for keyboard input and screen output.⁶¹

8.3 The dup2() System Call: The Key to Redirection

The mechanism that enables I/O redirection is the dup2(int oldfd, int newfd) system call. Its function is to duplicate a file descriptor, but with a crucial twist: it makes newfd a copy of oldfd, but forces the new descriptor's number to be newfd.⁶² The process is atomic:

1. If newfd is already an open file descriptor, the kernel silently closes it.
2. The kernel then makes newfd a copy of oldfd. Both oldfd and newfd now refer to the same underlying open file entry in the kernel, sharing the same file offset and status flags.⁶²

This ability to overwrite a specific file descriptor (like 1 for stdout) with a copy of another (like one pointing to a disk file) is the key to redirection.⁶⁴

8.4 Implementing Redirection (> and <)

Redirection is an act of deception orchestrated by the shell. The programs being executed (ls, grep, etc.) are typically written to be oblivious to redirection; they simply read from stdin (fd 0) and write to stdout (fd 1). The shell leverages the window of opportunity between fork() and execvp() to manipulate the child's file descriptor table, tricking the new program into reading from or writing to a file instead of the terminal.

The sequence for output redirection (e.g., ls -l > output.txt) is as follows:

1. The parent shell parses the command and identifies the > operator and the target filename output.txt.

2. The parent shell calls `fork()`.
3. Inside the child process:
 - a. The child opens the file `output.txt` for writing using the `open()` system call. This returns a new file descriptor, let's say `fd = 3`.
 - b. The child then calls `dup2(3, 1)`. This is the critical step. The kernel closes the original `stdout` (`fd 1`, which pointed to the terminal) and makes `fd 1` a duplicate of `fd 3`. Now, any write operation directed at file descriptor `1` will be sent to `output.txt`.
 - c. The original file descriptor `3` is now redundant and should be closed with `close(3)`.
 - d. Finally, the child calls `execvp("ls",...)`. The `ls` program starts, completely unaware of this manipulation. It performs its work and writes its output to its standard output (file descriptor `1`), which the kernel now routes to `output.txt`.

Input redirection (`<`) follows a similar pattern, but involves opening a file for reading and using `dup2(fd, 0)` to overwrite `stdin`.

8.5 Implementing Pipes (I)

Pipes extend the concept of redirection from files to processes, allowing the `stdout` of one command to be connected directly to the `stdin` of another. A pipe is a unidirectional, in-memory buffer managed by the kernel, behaving like an anonymous temporary file.²⁵

The `pipe(int pipefd)` system call creates a pipe and provides two file descriptors in the integer array `pipefd`: `pipefd[0]` is the read end of the pipe, and `pipefd[1]` is the write end.²⁵

The complex sequence for a command like `cmd1 | cmd2` is a masterful coordination of two child processes and a pipe²⁵:

1. The parent shell calls `pipe(pipefd)` to create the pipe.
2. The shell calls `fork()` to create the first child (for `cmd1`).
3. In Child 1 (`cmd1`):
 - a. It will write to the pipe, so it does not need the read end. It calls `close(pipefd[0])`.
 - b. It redirects its `stdout` to the write end of the pipe by calling `dup2(pipefd[1], 1)`.
 - c. It closes the now-redundant original write end, `close(pipefd[1])`.
 - d. It calls `execvp()` to run `cmd1`. All of `cmd1`'s output will now flow into the pipe.
4. The shell calls `fork()` again to create the second child (for `cmd2`).
5. In Child 2 (`cmd2`):
 - a. It will read from the pipe, so it does not need the write end. It calls `close(pipefd[1])`.
 - b. It redirects its `stdin` from the read end of the pipe by calling `dup2(pipefd[0], 0)`.
 - c. It closes the now-redundant original read end, `close(pipefd[0])`.
 - d. It calls `execvp()` to run `cmd2`. `cmd2` will read its input from the pipe, receiving `cmd1`'s output.

6. In the Parent Shell:
 - a. The parent does not read from or write to the pipe itself. It must close both ends, `close(pipefd)` and `close(pipefd)`. This is crucial; the read end of the pipe will not signal EOF to the reading process (`cmd2`) until all write ends have been closed.
 - b. The parent calls `waitpid()` twice to reap both children.

Section 9: Handling Interruptions: A Primer on Signal Management

9.1 What are Signals?

Signals are a fundamental mechanism in Unix-like systems for handling asynchronous events. They serve as a form of lightweight, software-level interrupt, notifying a process that a specific event has occurred.⁶⁶ Each signal is identified by a unique integer and a symbolic name (e.g.,

`SIGINT` for signal 2) and is generated in response to various conditions, such as a hardware exception, a kernel notification, or a user action.⁶⁶

For an interactive shell, the most relevant signals are those generated by keyboard input:

- `SIGINT` (Signal Interrupt): This signal is typically sent to the foreground process when the user presses `Ctrl+C`. The default action for a process receiving `SIGINT` is to terminate immediately.⁶⁸
- `SIGTSTP` (Signal Terminal Stop): This signal is sent when the user presses `Ctrl+Z`. The default action is to suspend (pause) the process's execution. The process remains in memory but is not scheduled for CPU time until it receives a `SIGCONT` signal.⁶⁸
- `SIGCHLD` (Signal Child): This signal is sent from the kernel to a parent process whenever one of its child processes terminates, stops, or is continued. It serves as an asynchronous notification that `waitpid()` can be called to reap the child.

9.2 Signal Dispositions

When a process receives a signal, it can respond in one of three ways, known as the signal's disposition ⁶⁶:

1. **Perform the Default Action:** The process can allow the kernel to perform the default behavior associated with the signal (e.g., terminate on SIGINT, stop on SIGTSTP).
2. **Ignore the Signal:** The process can instruct the kernel to simply discard the signal, taking no action. The SIGKILL and SIGSTOP signals are special exceptions that cannot be ignored or caught.
3. **Catch the Signal:** The process can register a custom function, called a **signal handler**, to be executed when the signal is received. This allows a program to define its own behavior for handling interruptions.

9.3 Installing Signal Handlers: `signal()` vs. `sigaction()`

There are two primary system calls for setting a signal's disposition:

- `signal()`: This is the original, simpler C standard library function. However, its use is now strongly discouraged. Its behavior has historically varied across different Unix implementations, particularly regarding whether a signal handler, once triggered, remains installed or is reset to the default. This inconsistency makes it unreliable for portable and robust applications.⁷⁰
- `sigaction()`: This is the modern, POSIX-standard function for managing signals. It is more complex but provides a far more reliable, portable, and powerful interface. It uses a struct `sigaction` to specify the handler, a set of signals to block during the handler's execution, and various flags to control behavior (e.g., `SA_RESTART` to automatically restart interrupted system calls).⁷⁰ For any serious systems programming, `sigaction()` is the correct and recommended choice.

A critical consideration when writing signal handlers is the concept of **async-signal-safety**. A signal can arrive at any moment, interrupting the main program's execution at an arbitrary point. If the handler then calls a function that is not reentrant (e.g., one that modifies a global data structure that the main program was also in the middle of modifying), a race condition can occur, leading to data corruption or a crash. Therefore, only a small subset of system calls and library functions, designated as "async-signal-safe" (like `write()`, but not `printf()`), can be safely called from within a signal handler.⁷² A common and safe practice in a handler is to do minimal work, such as setting a global

volatile `sig_atomic_t` flag, which the main program's loop can then check and act upon later.

9.4 Signal Handling Logic in a Shell

The logic for handling signals in a shell is nuanced because the shell must act as a manager for other processes. The correct behavior depends on whether the shell itself or a child process is the intended recipient of the signal. This is managed by the kernel through the concept of **process groups** and the **foreground process group**. The terminal directs keyboard-generated signals (Ctrl+C, Ctrl+Z) only to the processes in the current foreground process group.⁶⁹ A sophisticated shell will place each job (pipeline of commands) into a new process group and use system calls to tell the terminal which group is in the foreground.

For a simple shell, the logic can be simplified as follows:

- **When the shell is at the prompt (interactive):** The shell itself is in the foreground.
 - It should **catch SIGINT**. The handler should not terminate the shell but should instead break out of any pending input read and redisplay the prompt, effectively canceling the current line.
 - It should **ignore SIGTSTP**. There is no concept of "stopping" the shell itself in this context.⁶⁹
- **When the shell is running a foreground command:** The shell has forked a child and is waiting for it. Both the shell and the child are in the foreground process group.
 - When Ctrl+C is pressed, both processes receive SIGINT. The child process, which has not had its signal handlers modified, will perform the default action and terminate. The parent shell, which has its own custom handler, will ignore the signal or handle it gracefully. The parent's waitpid() call will then return, indicating the child was terminated by a signal, and the shell can print a new prompt.
 - This ensures that user interruptions correctly target the running command, not the shell that launched it, providing the expected interactive experience.

Conclusion

This guide has provided a comprehensive theoretical blueprint for constructing a command-line shell in a Linux environment. By deconstructing the shell into its constituent parts, we have explored the fundamental principles of process management, inter-process communication, and system-level interaction that define the relationship between a user and the operating system kernel.

The journey begins with understanding the shell's architectural role as a REPL-based command interpreter, a user-space application that translates human commands into kernel-executable system calls. The core of its functionality rests upon the canonical

fork-exec-wait cycle. The `fork()` system call provides the mechanism for process creation through cloning, while the `execvp()` function enables process transformation, replacing the cloned process with a new program. The `waitpid()` call is essential for parent-child synchronization, ensuring orderly execution and preventing the accumulation of "zombie" processes—a state revealed not as an error, but as a necessary mechanism for asynchronous communication of a child's exit status.

The guide has illuminated the critical distinction between external and built-in commands, a distinction driven by the non-negotiable principle of process isolation. Commands that must alter the shell's own internal state, such as `cd`, must be implemented as built-ins because a child process cannot modify its parent's environment. Furthermore, the powerful I/O redirection capabilities of a shell are shown to be an elegant deception. By manipulating file descriptors with `dup2()` and establishing communication channels with `pipe()` in the brief window between `fork()` and `exec()`, the shell orchestrates complex data flows between processes that remain entirely unaware of the manipulation.

Finally, the principles of asynchronous signal handling demonstrate the shell's role as a robust session manager, capable of gracefully handling user interruptions like `SIGINT` and `SIGTSTP`. The preference for `sigaction()` over `signal()` underscores the importance of reliability and portability in systems programming.

Building a shell is more than a simple coding exercise; it is a practical and profound exploration of core operating systems theory. It forces the developer to engage directly with the primitives of process lifecycle, memory isolation, I/O streams, and asynchronous events. By following the incremental development path outlined—from a basic REPL to a feature-rich interpreter with redirection and signal handling—an aspiring systems programmer can synthesize scattered knowledge into a cohesive and deep understanding of how a modern operating system works from the user's perspective. The ultimate deliverable is not just a functional program, but a tangible and hard-won intuition for the intricate dance of processes that underlies all computation on a Linux system.

Works cited

1. The Linux Shell and Commands, accessed on October 5, 2025, <https://www.cs.dartmouth.edu/campbell/cs50/shell.html>
2. The Shell Command Interpreter - the Learn Linux Project, accessed on October 5, 2025, <https://www.learnlinux.org.za/courses/build/fundamentals/ch04s04.html>
3. What is Shell? | DataCamp, accessed on October 5, 2025, <https://www.datacamp.com/blog/what-is-shell>
4. Shell script - Wikipedia, accessed on October 5, 2025, https://en.wikipedia.org/wiki/Shell_script
5. What is a command shell? - PowerShell | Microsoft Learn, accessed on October 5, 2025, <https://learn.microsoft.com/en-us/powershell/scripting/what-is-a-command-shell>

[?view=powershell-7.5](#)

6. Read-eval-print loop - Wikipedia, accessed on October 5, 2025, https://en.wikipedia.org/wiki/Read%E2%80%93eval%E2%80%93print_loop
7. Architecture of Linux operating system The architecture of Linux is composed of kernel, shell and application programs that is s, accessed on October 5, 2025, <https://smmcollege.in/wp-content/uploads/2021/01/Architecture-of-Linux-operating-system.pdf>
8. The Command Shell - University of Iowa, accessed on October 5, 2025, <https://homepage.stat.uiowa.edu/~luke/classes/STAT4580-2024/shell.html>
9. Architecture of Linux Operating System - GeeksforGeeks, accessed on October 5, 2025, <https://www.geeksforgeeks.org/linux-unix/architecture-of-linux-operating-system/>
10. Linux Architecture - Medium, accessed on October 5, 2025, <https://medium.com/@pragnapcu/linux-architecture-ed3ed126b58d>
11. Understanding the Architecture of the Linux Operating System - YouStable, accessed on October 5, 2025, <https://www.youstable.com/blog/architecture-of-the-linux-operating-system/>
12. Evolution of shells in Linux - IBM Developer, accessed on October 5, 2025, <https://developer.ibm.com/tutorials/l-linux-shells/>
13. List of command-line interpreters - Wikipedia, accessed on October 5, 2025, https://en.wikipedia.org/wiki/List_of_command-line_interpreters
14. Internal and External Commands in Linux - GeeksforGeeks, accessed on October 5, 2025, <https://www.geeksforgeeks.org/linux-unix/internal-and-external-commands-in-linux/>
15. 1.7. Using or Replacing Built-ins and External Commands - bash Cookbook [Book] - Sign In, accessed on October 5, 2025, <https://learning.oreilly.com/library/view/bash-cookbook/0596526784/ch01s09.html>
16. How to Structure C Projects: These Best Practices Worked for Me | Blog - Luca Cavallin, accessed on October 5, 2025, <https://www.lucavall.in/blog/how-to-structure-c-projects-my-experience-best-practices>
17. Create a C++ makefile project in Visual Studio | Microsoft Learn, accessed on October 5, 2025, <https://learn.microsoft.com/en-us/cpp/build/reference/creating-a-makefile-project?view=msvc-170>
18. Makefile Tutorial By Example, accessed on October 5, 2025, <https://makefiletutorial.com/>
19. Creating A Basic Make File for Compiling C Code, accessed on October 5, 2025, <https://www.codeproject.com/articles/Creating-A-Basic-Make-File-for-Compiling-C-Code>
20. A Simple Makefile Tutorial - Colby Computer Science, accessed on October 5, 2025, <https://www.cs.colby.edu/maxwell/courses/tutorials/maketutor/>

21. Incremental Unix Shell Development - C2 wiki, accessed on October 5, 2025, <https://wiki.c2.com/?IncrementalUnixShellDevelopment>
22. Programming Tips: Incremental Development and Scaffolding - Linux Geekoid, accessed on October 5, 2025, <https://linuxgeekoid.wordpress.com/2011/06/18/programming-tips-incremental-development/>
23. Tutorial - Write a Shell in C - Stephen Brennan, accessed on October 5, 2025, <https://brennan.io/2015/01/16/write-a-shell-in-c/>
24. Build your own Shell : PART 1 - DEV Community, accessed on October 5, 2025, <https://dev.to/angelotheman/build-your-own-shell-part-1-1192>
25. Making your own Linux Shell in C - GeeksforGeeks, accessed on October 5, 2025, <https://www.geeksforgeeks.org/c/making-linux-shell-c/>
26. Let's Write a Simple Shell in C! - YouTube, accessed on October 5, 2025, <https://www.youtube.com/watch?v=YMEHrXSsdo0>
27. [C] Adding functionality to a custom C shell : r/learnprogramming - Reddit, accessed on October 5, 2025, https://www.reddit.com/r/learnprogramming/comments/4c3dgr/c_adding_functionality_to_a_custom_c_shell/
28. What Is REPL? - DigitalOcean, accessed on October 5, 2025, <https://www.digitalocean.com/community/tutorials/what-is-repl>
29. The Python Standard REPL: Try Out Code and Ideas Quickly, accessed on October 5, 2025, <https://realpython.com/python-repl/>
30. A Beginner's Guide to Writing a Simple Shell Script in C | by yongjoon - Medium, accessed on October 5, 2025, <https://medium.com/@avocadi/creating-my-own-shell-for-fun-635ea0d3f7b7>
31. Shell 1 (C-Shell) - Brown Computer Science, accessed on October 5, 2025, <https://cs.brown.edu/courses/cs033/shell1/>
32. COMP 530: Lab 1: Parser for a Shell, accessed on October 5, 2025, <https://www.cs.unc.edu/~porter/courses/comp530/f23/lab1.html>
33. Execute a Program: the execvp() System Call - Csl.mtu.edu, accessed on October 5, 2025, <https://www.csl.mtu.edu/cs4411.ck/www/NOTES/process/fork/exec.html>
34. Parse.c, accessed on October 5, 2025, <https://people.clarkson.edu/class/cs444/labs/lab01/parse.html>
35. Project C-Shell - Brown CS, accessed on October 5, 2025, <https://cs.brown.edu/courses/cs033/docs/proj/shell1.pdf>
36. String Tokenization in C - GeeksforGeeks, accessed on October 5, 2025, <https://www.geeksforgeeks.org/cpp/string-tokenization-in-c/>
37. How to tokenize a string in C++ - Educative.io, accessed on October 5, 2025, <https://www.educative.io/answers/how-to-tokenize-a-string-in-cpp>
38. C library - strtok() function - Tutorials Point, accessed on October 5, 2025, https://www.tutorialspoint.com/c_standard_library/c_function_strtok.htm
39. C++ cstring strtok() - Tokenize String - Vultr Docs, accessed on October 5, 2025, <https://docs.vultr.com/cpp/standard-library/cstring/strtok>
40. strtok() — Tokenize String - IBM, accessed on October 5, 2025, <https://www.ibm.com/docs/en/i/7.5.0?topic=functions-strtok-tokenize-string>

41. What is the `execvp()` function in C? - Educative.io, accessed on October 5, 2025, <https://www.educative.io/answers/what-is-the-execvp-function-in-c>
42. How to use `execvp()` in C and C++ - IONOS, accessed on October 5, 2025, <https://www.ionos.com/digitalguide/websites/web-development/execvp/>
43. How to use a `fork()` system call - Educative.io, accessed on October 5, 2025, <https://www.educative.io/answers/how-to-use-a-fork-system-call>
44. Operating Sytem: `fork()` system call | by Ducanh03 - Medium, accessed on October 5, 2025, <https://medium.com/@namobilegame/operating-sytem-fork-system-call-0b85c0dc96be>
45. The `fork()` System Call - Csl.mtu.edu, accessed on October 5, 2025, <https://www.csl.mtu.edu/cs4411.ck/www/NOTES/process/fork/create.html>
46. `Fork()` System Call | Scaler Topics, accessed on October 5, 2025, <https://www.scaler.com/topics/fork-system-call/>
47. `execvp` - POSIX Function Reference - SAS Support, accessed on October 5, 2025, <https://support.sas.com/documentation/onlinedoc/sasc/doc700/html/lr2/zid-7307.htm>
48. What is the difference between internal and external commands in Linux? - Web Hosting Technical Support Discussion Forum, accessed on October 5, 2025, <https://www.accuwebhosting.com/discussion/forum/peer-support/linux-hosting/35609-what-is-the-difference-between-internal-and-external-commands-in-linux>
49. Why isn't `cd` in `/bin` ? : `r/linux` - Reddit, accessed on October 5, 2025, https://www.reddit.com/r/linux/comments/8unz8/why_isnt_cd_in_bin/
50. Shell builtin - Wikipedia, accessed on October 5, 2025, https://en.wikipedia.org/wiki/Shell_builtin
51. `Cd` (command) - Wikipedia, accessed on October 5, 2025, [https://en.wikipedia.org/wiki/Cd_\(command\)](https://en.wikipedia.org/wiki/Cd_(command))
52. C shell built-in commands list - IBM, accessed on October 5, 2025, https://www.ibm.com/docs/ro/ssw_aix_71/osmanagement/c_builtin_cmd_list.html
53. `Wait` & `Waitpid`, accessed on October 5, 2025, <https://webdocs.cs.ualberta.ca/~tony/C379/C379Labs/Lab3/wait.html>
54. `wait()` vs `waitpid()` in C - Educative.io, accessed on October 5, 2025, <https://www.educative.io/answers/wait-vs-waitpid-in-c>
55. `waitpid()` — Wait for a Specific Child Process to End - IBM, accessed on October 5, 2025, <https://www.ibm.com/docs/en/zvm/7.3.0?topic=descriptions-waitpid-wait-specific-child-process-end>
56. Zombie and Orphan Process in OS | Scaler Topics, accessed on October 5, 2025, <https://www.scaler.com/topics/operating-system/zombie-and-orphan-process-in-os/>
57. Zombie process vs Orphan process - Stack Overflow, accessed on October 5, 2025, <https://stackoverflow.com/questions/20688982/zombie-process-vs-orphan-process>
58. Difference between Zombie, Orphan and Daemon Processes - GeeksforGeeks,

- accessed on October 5, 2025,
<https://www.geeksforgeeks.org/operating-systems/difference-between-zombie-orphan-and-daemon-processes/>
59. medium.com, accessed on October 5, 2025,
<https://medium.com/@dhar.ishan04/here-is-all-you-need-to-know-about-file-descriptors-in-linux-d93f05166026#:~:text=If%20everything%20behaves%20like%20a,for%20these%20files%20or%20resources.>
60. What is a File Descriptor & How to Close It - Lenovo, accessed on October 5, 2025, <https://www.lenovo.com/us/en/glossary/file-descriptor/>
61. Using file descriptors - IBM, accessed on October 5, 2025,
https://www.ibm.com/docs/ssw_aix_71/com.ibm.aix.genprogc/using_file_descriptors.htm
62. dup (system call) - Wikipedia, accessed on October 5, 2025,
[https://en.wikipedia.org/wiki/Dup_\(system_call\)](https://en.wikipedia.org/wiki/Dup_(system_call))
63. dup(2) - Linux manual page - man7.org, accessed on October 5, 2025,
<https://man7.org/linux/man-pages/man2/dup.2.html>
64. dup() and dup2() Linux system call - GeeksforGeeks, accessed on October 5, 2025, <https://www.geeksforgeeks.org/c/dup-dup2-linux-system-call/>
65. Solved: Understanding dup2 (C System Call) - Experts Exchange, accessed on October 5, 2025,
<https://www.experts-exchange.com/questions/29120489/Understanding-dup2-C-System-Call.html>
66. Signal Handling In Linux Through The signal() Function - GeeksforGeeks, accessed on October 5, 2025,
<https://www.geeksforgeeks.org/linux-unix/signal-handling-in-linux-through-the-signal-function/>
67. Writing a shell in C, how do I use signal handling? : r/C_Programming - Reddit, accessed on October 5, 2025,
https://www.reddit.com/r/C_Programming/comments/772qhf/writing_a_shell_in_c_how_do_i_use_signal_handling/
68. Shell Scripting - How to send Signal to a Processes - GeeksforGeeks, accessed on October 5, 2025,
<https://www.geeksforgeeks.org/linux-unix/shell-scripting-how-to-send-signal-to-a-processes/>
69. Signals (Bash Reference Manual) - GNU, accessed on October 5, 2025,
https://www.gnu.org/software/bash/manual/html_node/Signals.html
70. What is the difference between sigaction and signal? - Stack Overflow, accessed on October 5, 2025,
<https://stackoverflow.com/questions/231912/what-is-the-difference-between-sigaction-and-signal>
71. SIG01-C. Understand implementation-specific details regarding signal handler persistence, accessed on October 5, 2025,
<https://wiki.sei.cmu.edu/confluence/display/c/SIG01-C.+Understand+implementation+specific+details+regarding+signal+handler+persistence>
72. Sending and Handling Signals in C (kill, signal, sigaction) - YouTube, accessed on

October 5, 2025, <https://www.youtube.com/watch?v=83M5-NPDeWs>
73. How can I handle SIGINT trap with a user prompt in shell script?, accessed on
October 5, 2025,
<https://unix.stackexchange.com/questions/240602/how-can-i-handle-sigint-trap-with-a-user-prompt-in-shell-script>